Autobahn BFT & Malachite 분석

작성일: 2025-01-07 **분석 대상**: Autobahn BFT, Malachite (CometBFT variant) **목적**: 최신 BFT 합의 알고리즘의 혁신과 CometBFT 진화 방향 분석

목차

1. Autobahn BFT
2. Malachite
3. 비교 분석
4. CometBFT 적용 가능성

---

1. Autobahn BFT

1.1 개요

Autobahn (2024, arxiv:2401.10369)는 **pipelined DAG-based BFT** 프로토콜입니다.

핵심 혁신:

1. **Pipelined DAG**: 데이터 전파와 합의 병렬화
2. **Zero-wait Commits**: 대기 없이 즉시 커밋
3. **Optimistic Path**: 정상 시 빠른 경로

1.2 아키텍처

```
┌──────────────────────────────────────────────────────────┐
│                    Autobahn Architecture                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────────────────────────────────────────┐   │
│  │              Pipelined Data Path                 │   │
│  │                                                  │   │
│  │   Round N   Round N+1   Round N+2   Round N+3    │   │
│  │     │          │           │           │         │   │
│  │     ▼          ▼           ▼           ▼         │   │
│  │   [Batch]──>[Batch]──>[Batch]──>[Batch]          │   │
│  │     ‖          ‖           ‖           ‖         │   │
│  │     ‖          ‖           ‖           ‖         │   │
│  └────‖──────────‖───────────‖───────────‖──────────┘   │
│       ‖          ‖           ‖           ‖              │
│       ▼          ▼           ▼           ▼              │
│  ┌──────────────────────────────────────────────────┐   │
│  │              Consensus Path                      │   │
│  │                                                  │   │
│  │   [Leader]──>[Vote]──>[QC]──>[Commit]            │   │
│  │                                                  │   │
│  └──────────────────────────────────────────────────┘   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.3 핵심 메커니즘

1.3.1 Pipelined Proposal

```
기존 (Sequential):
  Round 1: [Propose] → [Vote] → [Commit] → Done
  Round 2:                                  [Propose] → [Vote] → ...

Autobahn (Pipelined):
  Round 1: [Propose] ────────────────────────────────────────
  Round 2:    [Propose] ─────────────────────────────────────
  Round 3:       [Propose] ──────────────────────────────────
  Round 4:          [Propose] ───────────────────────────────
           └─[Vote]──────────────────────────────────────────┘
                    └─[QC]──────────────────────────────────┘
                             └─[Commit]────────────────────┘
```

의사코드:

```
// Autobahn Pipelined Consensus

on receive proposal(round, block):
    if round > currentRound:
        // 파이프라인 - 이전 라운드 처리와 동시 진행
        pipeline.add(round, block)

    // 이전 라운드들의 투표 처리
    for r in pendingRounds:
        if hasQuorum(r):
            advanceRound(r)

function advanceRound(round):
    qc = createQC(votes[round])

    // Zero-wait: QC 있으면 즉시 커밋
    if qc.round >= commitRound + 2:
        commit(qc.block)

    broadcast(qc)
```

1.3.2 Optimistic Fast Path

```
정상 케이스 (f=0):
  - 1 라운드 만에 커밋
  - 모든 노드 응답

비정상 케이스:
  - Fallback to standard path
  - 3 라운드 커밋
```

1.4 성능 특성

메트릭	값
메시지 복잡도	O(n)
Optimistic 지연	1 라운드
Pessimistic 지연	3 라운드
처리량	~200,000 TPS (이론)

1.5 vs HotStuff 비교

특성	HotStuff	Autobahn
파이프라이닝	Chained (블록 간)	전체 (라운드 간)
커밋 지연	3 라운드	1-3 라운드
복잡도	중간	높음
Optimistic path	없음	있음

---

2. Malachite

2.1 개요

Malachite는 Informal Systems가 개발하는 **CometBFT의 Rust 재구현 및 개선**입니다.

Repository: https://github.com/informalsystems/malachite

목표:

1. Rust로 안전성 향상
2. 모듈화된 아키텍처
3. 정형 검증 (Formal Verification)
4. 성능 최적화

2.2 아키텍처

```
┌─────────────────────────────────────────────────────────┐
│                  Malachite Architecture                  │
├─────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │                   Application                     │    │
│  │                  (via Driver)                     │    │
│  └──────────────────────────────────────────────────┘    │
│                          ▲                               │
│                          │ Callbacks                     │
│                          ▼                               │
│  ┌──────────────────────────────────────────────────┐    │
│  │                 Consensus Engine                  │    │
│  │                                                   │    │
│  │   ┌──────────┐   ┌──────────┐   ┌──────────┐     │    │
│  │   │  State   │   │  Round   │   │  Voting  │     │    │
│  │   │ Machine  │   │ Manager  │   │  Logic   │     │    │
│  │   └──────────┘   └──────────┘   └──────────┘     │    │
│  └──────────────────────────────────────────────────┘    │
│                          ▲                               │
│                          │ Messages                      │
│                          ▼                               │
│  ┌──────────────────────────────────────────────────┐    │
│  │                  Network Layer                    │    │
│  │                (Gossip Protocol)                  │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
└─────────────────────────────────────────────────────────┘
```

2.3 CometBFT와의 차이점

2.3.1 언어 및 안전성

```
CometBFT (Go):
- 런타임 에러 가능
- GC 오버헤드
- Data race 가능

Malachite (Rust):
- 컴파일 타임 안전성
- 제로 오버헤드 추상화
- 메모리 안전 보장
```

2.3.2 모듈화

```
// Malachite의 모듈화된 구조

pub trait Consensus {
    type Value;
    type Vote;
    type Proposal;

    fn propose(&self, height: Height, round: Round) -> Self::Proposal;
    fn vote(&self, proposal: &Self::Proposal) -> Self::Vote;
    fn decide(&self, value: Self::Value);
}

pub trait Network {
    fn broadcast(&self, message: Message);
    fn receive(&self) -> Option<Message>;
}

pub trait Storage {
    fn save_state(&self, state: &ConsensusState);
    fn load_state(&self) -> ConsensusState;
}

// 조립
pub struct MalachiteNode<C, N, S>
where
    C: Consensus,
    N: Network,
    S: Storage,
{
    consensus: C,
    network: N,
    storage: S,
}
```

2.3.3 정형 검증

```
TLA+ 스펙:
- Safety 증명
- Liveness 증명 (부분 동기 가정)
- 불변식 검증

Model Checking:
- 모든 가능한 상태 탐색
- 버그 사전 발견
```

2.4 핵심 개선사항

2.4.1 PBTS (Proposer-Based Timestamp) 개선

```
// Malachite의 PBTS 구현

pub struct ProposerBasedTimestamp {
    precision: Duration,
    message_delay: Duration,
}

impl ProposerBasedTimestamp {
    pub fn is_timely(&self, proposal: &Proposal, receive_time: Instant) -> bool {
        let timestamp = proposal.timestamp;
        let lower_bound = timestamp - self.precision;
        let upper_bound = timestamp + self.message_delay + self.precision;

        receive_time >= lower_bound && receive_time <= upper_bound
    }
}
```

개선점:

- 더 정밀한 시간 동기화
- 네트워크 지연 적응
- Byzantine 시간 공격 방어

2.4.2 Vote Extensions

```rust
// 확장된 투표 구조

pub struct ExtendedVote {
    // 기본 투표
    pub vote_type: VoteType,
    pub height: Height,
    pub round: Round,
    pub block_hash: Option<Hash>,

    // 확장 데이터 (애플리케이션 정의)
    pub extension: Vec<u8>,
    pub extension_signature: Signature,
}

// 사용 예: Oracle 가격 데이터
pub struct OracleExtension {
    pub prices: HashMap<Asset, Price>,
    pub timestamp: Instant,
}
```

활용:

- 오라클 데이터
- 크로스체인 증명
- 랜덤 비콘

2.5 성능 개선

```
벤치마크 (예상):

                CometBFT      Malachite
TPS (4노드):     ~10,000      ~15,000
TPS (100노드):   ~1,000       ~2,000
메모리:          중간          낮음
CPU:            중간          낮음
```

최적화 기법:

1. **Zero-copy 메시지**: 불필요한 복사 제거
2. **Lock-free 자료구조**: 경합 감소
3. **Batch 처리**: 네트워크 효율성
4. **SIMD 서명 검증**: 병렬 암호 연산

---

3. 비교 분석

3.1 알고리즘 비교

특성	CometBFT	Autobahn	Malachite
기반	Tendermint	DAG + Pipeline	Tendermint
언어	Go	- (논문)	Rust
메시지 복잡도	$O(n^2)$	$O(n)$	$O(n^2)$
커밋 지연	3단계	1-3단계	3단계
파이프라이닝	제한적	전체	개선된
정형 검증	부분	없음	전체

3.2 성능 비교

```
처리량 (TPS) - 4 노드:

CometBFT:   ██████████████████ 10,000
Malachite:  ═══════════════════════════ 15,000
Autobahn:   ═══════════════════════════════════════════════════ 50,000+

커밋 지연 (ms):

Autobahn:   ═══ 100
Malachite:  ═══════════ 300
CometBFT:   ═══════════════ 500
```

3.3 적용 난이도

측면	CometBFT	Autobahn	Malachite
학습 곡선	낮음	높음	중간
마이그레이션	N/A	어려움	쉬움 (호환)
문서화	좋음	제한적	중간
커뮤니티	큼	없음	성장 중
프로덕션 검증	높음	없음	낮음

4. CometBFT 적용 가능성

4.1 단기 개선 (Malachite 아이디어)

PBTS 최적화

```go
// 현재 CometBFT
func (cs *State) proposalIsTimely() bool {
    sp := cs.state.ConsensusParams.Synchrony.InRound(cs.Proposal.Round)
    return cs.Proposal.IsTimely(cs.ProposalReceiveTime, sp)
}

// 개선안: 적응형 PBTS
type AdaptivePBTS struct {
    precision       time.Duration
    messageDelay    time.Duration
    networkLatency  *MovingAverage  // 동적 측정
}

func (p *AdaptivePBTS) IsTimely(proposal *Proposal, receiveTime time.Time) bool {
    // 네트워크 상태에 따라 동적 조정
    adjustedDelay := p.messageDelay + p.networkLatency.Average()

    lowerBound := proposal.Timestamp.Add(-p.precision)
    upperBound := proposal.Timestamp.Add(adjustedDelay + p.precision)

    return receiveTime.After(lowerBound) && receiveTime.Before(upperBound)
}
```

Vote Extension 활용

```go
// 현재: 기본 투표만
type Vote struct {
    Type             VoteType
    Height           int64
    Round            int32
    BlockID          BlockID
    Timestamp        time.Time
    ValidatorAddress Address
    ValidatorIndex   int32
    Signature        []byte
}

// 개선안: 확장 투표
type ExtendedVote struct {
    Vote
    Extension          []byte  // 애플리케이션 데이터
    ExtensionSignature []byte
}

// 사용 예: MEV 방지
type MEVProtectionExtension struct {
    TxOrderCommitment []byte  // 트랜잭션 순서 커밋먼트
    RandomnessShare   []byte  // 분산 랜덤 생성
}
```

4.2 중기 개선 (Autobahn 아이디어)

파이프라이닝

```go
// 현재: 순차 처리
func (cs *State) receiveRoutine() {
    for {
        select {
        case msg := <-cs.peerMsgQueue:
            cs.handleMsg(msg)  // 하나씩 처리
        }
    }
}

// 개선안: 파이프라인 처리
type PipelinedState struct {
    proposalPipeline chan *Proposal  // Proposal 파이프라인
    votePipeline     chan *Vote      // Vote 파이프라인
    commitPipeline   chan *Commit    // Commit 파이프라인
}

func (ps *PipelinedState) run() {
    go ps.processProposals()
    go ps.processVotes()
    go ps.processCommits()

    // 각 파이프라인이 독립적으로 진행
}

func (ps *PipelinedState) processProposals() {
    for proposal := range ps.proposalPipeline {
        // 다음 라운드 proposal 처리
        // 이전 라운드 완료 대기 없음
        ps.handleProposal(proposal)
    }
}
```

Optimistic Fast Path

```go
// 정상 케이스 빠른 경로
type OptimisticConsensus struct {
    fastPath bool
    votes    map[int32]map[Address]*Vote
}

func (oc *OptimisticConsensus) tryFastCommit(height int64, round int32) bool {
    // 모든 validator가 같은 블록에 투표했는지 확인
    votes := oc.votes[round]
    if len(votes) == oc.totalValidators {
        blockHash := votes[0].BlockID.Hash
        allSame := true
        for _, v := range votes {
            if !bytes.Equal(v.BlockID.Hash, blockHash) {
                allSame = false
                break
            }
        }
        if allSame {
            // Fast path: 1 라운드 커밋!
            return oc.fastCommit(height, round, blockHash)
        }
    }
    return false  // Fallback to standard path
}
```

4.3 장기 개선 (DAG 기반)

```go
// DAG 구조 통합 (실험적)
type DAGConsensus struct {
    dag      *DAG
    vertices map[Round]map[Address]*Vertex
}

type Vertex struct {
    Round    Round
    Author   Address
    Parents  []VertexID  // 이전 라운드 참조
    Payload  []Transaction
    QC       *QuorumCert
}

func (dc *DAGConsensus) proposeVertex(round Round) *Vertex {
    // 이전 라운드의 2f+1 vertices 참조
    parents := dc.selectParents(round - 1)

    return &Vertex{
        Round:   round,
        Author:  dc.address,
        Parents: parents,
        Payload: dc.mempool.GetBatch(),
    }
}

func (dc *DAGConsensus) orderDAG() []Transaction {
    // DAG를 선형 순서로 정렬
    var ordered []Transaction
    for _, anchor := range dc.anchors {
        history := dc.getCausalHistory(anchor)
        for _, v := range topologicalSort(history) {
            ordered = append(ordered, v.Payload...)
        }
    }
    return ordered
}
```

4.4 구현 로드맵

```
Phase 1 (3-6개월): PBTS 및 Vote Extension 최적화
  └ Malachite의 아이디어 적용
  └ 호환성 유지

Phase 2 (6-12개월): 파이프라이닝 도입
  └ Autobahn의 파이프라인 개념
  └ Optimistic fast path

Phase 3 (12-18개월): DAG 통합 실험
  └ 하이브리드 구조
  └ 점진적 마이그레이션
```

요약

핵심 발견사항

1. **Autobahn**:
   - 파이프라이닝으로 처리량 극대화
   - Optimistic path로 지연 시간 최소화
   - 아직 실험적 단계
2. **Malachite**:
   - Rust로 안전성 향상
   - 정형 검증으로 신뢰성
   - CometBFT와 호환성
3. **CometBFT 개선 방향**:
   - 단기: PBTS, Vote Extension
   - 중기: 파이프라이닝
   - 장기: DAG 통합

권장사항

- **신규 프로젝트**: Malachite 고려 (성숙 시)
- **기존 프로젝트**: CometBFT + 점진적 개선
- **연구 프로젝트**: Autobahn 아이디어 실험

작성 완료: 2025-01-07 **총 페이지**: ~12페이지 **참고 자료**: Autobahn 논문 (arxiv:2401.10369), Malachite 레포지토리